

# 数据库管理系统课程大作业

## 项目完成报告

项目名称：基于 MongoDB 的问卷系统开发

阶段：第二阶段

|      |                                   |
|------|-----------------------------------|
| 课程名称 | 数据库管理系统                           |
| 技术栈  | Node.js + Express + MongoDB + EJS |
| 数据库  | MongoDB (Mongoose 建模)             |
| 提交内容 | 第二阶段项目完成报告                        |

说明：本报告依据第二阶段代码实现、MongoDB 设计、浏览器级验证与测试文档整理而成。

## 摘要

本项目在第一阶段在线问卷系统的基础上，按照课程给出的需求变更继续迭代，完成了第二阶段实现。系统仍然采用 Node.js、Express、EJS 与 MongoDB 作为核心技术栈，但数据模型和业务边界已经从“以问卷为中心”扩展为“问卷 + 独立题库 + 题目版本链”的组合结构。新增能力包括题目独立创建、题目共享、题目版本化、题库分组管理、跨问卷统计以及已发布问卷的快照锁定保护。

在数据设计上，第二阶段的核心调整是把题目从 Survey 内部的单一嵌入定义，扩展为 Question 集合中的独立版本实体，并使用 rootQuestionId 与 parentQuestionId 组织版本链；同时保留 Survey.questions 中的题目快照，从而保证已发布问卷不会因为后续题目编辑而发生历史污染。为了支持题目分组与复用，系统另外引入 QuestionLibrary 集合，统一维护题库和题目根版本之间的关联关系。

在业务实现上，系统新增了题目详情页、题目编辑页、题库主页、问卷构建器右侧题库面板，以及查看题目被哪些问卷使用和跨问卷统计的能力。浏览器侧对导入题库题后的字段做了只读保护，后端在响应记录中保存 sourceQuestionId、sourceQuestionRootId、sourceQuestionVersion 等来源字段，以保证统计维度准确。项目随后又完成了一轮启动验证、浏览器级手工检查和问题修复，确认第二阶段关键链路已经跑通。

总体来看，第二阶段没有推倒重做，而是在第一阶段保留的 version、status、jumpRules、visitedQuestionIds 等字段基础上进行增量设计，体现了 MongoDB 文档模型在课程项目迭代场景中的适配性。关键词：MongoDB；问卷系统；题库；版本链；快照锁定；跨问卷统计。

## 1. 项目目标

### 1.1 课题背景

在线问卷系统在课程调查、活动报名、满意度收集、信息登记等场景中都有较高的使用频率。第一阶段已经证明，问卷系统并不只是一个页面收集工具，后端需要同时解决题目建模、跳转控制、答案校验、统计分析和提交存储等问题，因此非常适合作为数据库管理系统课程的大作业课题。

进入第二阶段后，课程要求不再满足于“单份问卷可运行”，而是强调系统对后续变化的承受能力。例如：题目需要脱离问卷独立存在并被多个问卷复用；题目发生修改时要能够保留历史版本；共享题目后其他用户也应能导入使用；问卷一旦发布，后续的题库编辑不能污染旧问卷内容。这些要求直接推动数据库设计从单文档问卷建模升级到更稳定的增量扩展方案。

### 1.2 第二阶段建设目标

- 在不推倒第一阶段系统的前提下，完成题目独立化、共享、版本化、题库管理与跨问卷统计。
- 保持问卷填写、跳转、校验、提交与统计等第一阶段核心链路继续可用，并保证旧功能不回退。
- 通过问卷题目快照与状态锁定机制，保护已发布问卷的历史一致性。
- 为后续继续扩展题目搜索、版本恢复、协作权限等能力保留清晰的数据接口。

### 1.3 与课程要求的对应关系

| 课程要求 | 第二阶段实现内容 | 说明 |
|------|----------|----|
|------|----------|----|

|            |                                             |                       |
|------------|---------------------------------------------|-----------------------|
| 题目可复用      | Question 独立实体 + 题库导入                        | 题目可脱离问卷独立维护，并被多个问卷导入。 |
| 题目可共享      | 按用户名共享根版本                                   | 共享不复制题目根标识，保证来源关系可追踪。 |
| 支持版本管理     | rootQuestionId + parentQuestionId + version | 可查看版本历史并从旧版本创建新版本。    |
| 已发布问卷不可被污染 | 问卷内保存题目快照 + 发布后锁定                           | 旧问卷继续显示导入时的版本内容。      |
| 可做跨问卷分析    | Response 保存来源字段                             | 按题目根版本聚合所有问卷中的回答结果。   |
| 浏览器级验证     | 启动验证 + 手工检查 + 修复后回归                         | 第二阶段关键页面流程已实测通过。      |

## 2. 系统设计

### 2.1 技术选型与原因

技术栈继续沿用第一阶段的 Node.js + Express + MongoDB + EJS。这样做的原因不是偷懒，而是因为第二阶段重点在于数据设计和业务模型演进，而不是更换框架。Express 路由与控制器结构便于继续承载页面和表单流程；MongoDB 的文档模型适合表达题目版本、问卷快照和响应答案的嵌套结构；EJS 则便于在同一项目中快速补充题库、版本和统计页面。

| 层次   | 技术/组件                           | 继续使用的原因                  |
|------|---------------------------------|--------------------------|
| 后端框架 | Node.js + Express               | 控制器和路由职责清晰，适合课程项目持续迭代。   |
| 数据库  | MongoDB + Mongoose              | 适合题目版本、问卷快照、统计来源字段等文档结构。 |
| 视图层  | EJS + 原生 JavaScript             | 便于快速实现题库页、构建器和填写页的表单交互。  |
| 会话管理 | express-session + connect-mongo | 继续用于登录态持久化和匿名重复提交限制。     |

### 2.2 总体架构

第二阶段的总体架构仍然是“页面渲染 + 路由 + 控制器 + 服务层 + MongoDB”，但系统中心从单一 Survey 文档转向了多集合协作。浏览器请求页面或表单接口后，Express 路由分发到对应控制器；控制器负责权限和流程判断；服务层负责构建问卷、清洗题目版本数据、执行跳转与校验、聚合统计结果；模型层最终读写 User、Question、QuestionLibrary、Survey、Response 等集合。

浏览器 / 表单 → Express Routes → Controllers → Services → MongoDB

### 2.3 模块划分

- 用户模块负责注册、登录、退出登录和当前用户加载。
- 题目模块负责独立题目创建、查看详情、共享和新版本创建。
- 题库模块负责题库创建、题目根版本加题/移除和可访问题目列表整理。
- 问卷模块负责草稿创建、编辑、发布、关闭以及问卷题目快照存储。
- 填写与统计模块负责动态路径计算、统一校验、提交保存和整卷/单题/跨问卷统计。

### 2.4 主要业务流程

注册与登录流程沿用第一阶段实现：用户提交用户名和密码后，后端校验基本合法性，注册时以 bcryptjs 对密码做哈希后保存到 User.passwordHash，登录后将 userId 写入 session。

题目创建流程是第二阶段新增主线：用户进入 Question Bank 页面后，可新建独立题目并保存为 Question 文档；若从已有题目生成新版本，则新文档会继承相同 rootQuestionId，并把当前题目标记为 parentQuestionId。

题库构建流程中，系统通过 QuestionLibrary 记录 ownerId、题库名和 questionRootIds。问卷构建页面会读取自己拥有或有权访问的题目版本列表，并支持直接导入到 builder 中。

问卷保存流程中，前端 builder.js 会把构建结果统一组装为 builderPayload，后端 surveyBuilderService 负责清洗问题、补齐 questionId、保留 sourceQuestionRootId/sourceQuestionVersion 等快照字段，然后整体写入 Survey。

填写与提交流程延续第一阶段做法，仍然根据 jumpRules 计算 visitedQuestionIds，但在第二阶段的 Response.answers[] 中同时记录题目的根版本与具体版本，使统计页和题目详情页都能做更准确的跨问卷聚合。

## 2.5 项目目录与职责

| 目录/文件                                  | 职责说明                                                     |
|----------------------------------------|----------------------------------------------------------|
| src/models/Question.js                 | 定义题目版本实体，承载 rootQuestionId、parentQuestionId、version 等字段。 |
| src/models/QuestionLibrary.js          | 定义题库分组和 questionRootIds。                                 |
| src/controllers/questionController.js  | 负责题目创建、详情、共享、新版本、题库操作等流程。                                |
| src/controllers/surveyController.js    | 负责问卷构建、导入题目快照、发布关闭、填写与统计。                                |
| src/services/questionCatalogService.js | 整理用户可访问题目和题库数据，供 builder 页面使用。                           |
| public/js/builder.js                   | 负责问卷构建器的前端保存、导入题锁定与 payload 同步。                          |
| views/questions/*                      | 题库首页、题目详情、题目编辑等第二阶段页面模板。                                 |

## 3. MongoDB 设计

### 3.1 为什么继续使用 MongoDB

第二阶段需求变更进一步证明 MongoDB 的适用性。问卷系统中的题目、选项、校验规则、跳转规则、快照版本和统计来源字段都具有明显的层级结构。如果强行拆分到大量关系表，不仅会引入更多 join，也会让“复制题目快照到问卷中”这类操作变得繁琐。MongoDB 的文档模型更适合同时承载独立题目实体和问卷内部快照这两种访问模式。

### 3.2 集合划分

| 集合    | 主要内容         | 设计目的           |
|-------|--------------|----------------|
| users | 账号、密码哈希、创建时间 | 保存身份信息，支持权限控制。 |

|                   |                     |                               |
|-------------------|---------------------|-------------------------------|
| questions         | 独立题目版本、共享与版本关系      | 支持题目复用、共享与版本链。                |
| questionlibraries | 题库分组和题目根版本列表        | 支持按题库组织问题集合。                  |
| surveys           | 问卷主体、题目快照、状态与跳转     | 作为问卷业务主文档，支持整体读取与整体编辑。        |
| responses         | 每次提交的答案、访问路径、来源版本字段 | 避免问卷膨胀，便于统计与重复提交判断。           |
| sessions          | 用户会话信息              | 由 connect-mongo 维护登录态和匿名提交状态。 |

### 3.3 questions 集合设计

questions 集合是第二阶段最关键的新集合。与第一阶段“题目只存在于问卷内部”不同，现在每个题目版本都有自己的文档。ownerId 标识拥有者，rootQuestionId 标识版本族，parentQuestionId 指向父版本，version 用于版本号展示，visibility 和 sharedWithUserIds 用于共享控制，title/description/type/options/validation 等字段保存题目本身的定义。这样既能支持版本链，也能按根版本统计题目的使用范围。

### 3.4 questionlibraries 与 surveys 的关系

QuestionLibrary 并不直接复制完整题目内容，而是存储 questionRootIds，这样题库天然绑定到题目的根版本而不是单个旧版本。问卷真正导入时，questionCatalogService 会把当前可访问的具体版本整理出来，前端选择某个版本后，后端再把该版本快照写入 Survey.questions[]。因此，题库承担的是“组织入口”的职责，而问卷承担的是“发布时冻结内容”的职责。

### 3.5 surveys 与 responses 的快照字段

在 Survey.questions[] 中，第二阶段额外保存了 sourceQuestionId、sourceQuestionRootId、sourceQuestionVersion、sourceLocked 等字段。sourceLocked 用于标识这是从题库导入并且定义已锁定的题目；title、options、validation、jumpRules 等字段则保存实际快照内容。

在 Response.answers[] 中，系统继续保存 questionId 和 questionType，同时新增 sourceQuestionId、sourceQuestionRootId、sourceQuestionVersion、questionTitleSnapshot 等来源字段。这样即使后续题目版本继续演进，也能按照历史提交时对应的题目版本做统计解释。

### 3.6 索引与可扩展性设计

原有的 User.username、Survey.slug、Survey.ownerId、Response.surveyId、Response.respondentId 等索引仍然保留。第二阶段则进一步依赖 Question.rootQuestionId、Question.ownerId、Question.sharedWithUserIds 等字段进行版本查询和共享访问控制。整体看，系统并未破坏第一阶段索引使用场景，而是在原有查询路径上追加了题目维度的可扩展性。

### 3.7 与关系数据库方案的对比

如果使用关系数据库实现第二阶段，往往需要 questions、question\_versions、question\_shares、question\_libraries、library\_items、survey\_questions、response\_answers 等更多表，并且既要维护版本关系，又要在问卷发布时复制一份快照数据。对于课程项目规模来说，这会显著抬高实现复杂度。MongoDB 允许系统同时保留“独立题目版本文档”和“问卷内部快照文档”，因此更符合本项目的读写特征和阶段式迭代节奏。

## 4. 第二阶段需求变更实现

### 4.1 题目独立化设计

题目独立化的关键不是把原有 `questions[]` 从 `Survey` 中删掉，而是新增一套可复用题目实体，再在问卷中保留快照。这样一来，题库中的题目可以持续演进，而问卷中的历史版本依然可追踪。用户能够先创建题目，再决定是否加入题库、是否共享、是否导入到某份问卷中。

### 4.2 题目共享设计

共享机制按用户名执行。拥有者在题目详情页输入对方用户名后，系统会把被共享用户的 `userId` 写入 `sharedWithUserIds`。被共享用户随后在自己的题库页和问卷构建页中都能看到该题目版本或该根版本下的可访问版本。这种设计相对简单，但足以体现“同一题目跨账号复用”的业务能力。

### 4.3 题目版本链设计

版本链通过 `rootQuestionId` 和 `parentQuestionId` 组织。当用户从某个题目详情页创建新版本时，系统复制原题主要结构并把 `version` 加 1，同时保留相同 `rootQuestionId`。题目详情页会展示 `Version History`，方便查看某个根版本下所有历史版本。这样既能支持“继续修订题目”，也能保留旧版本被旧问卷引用的事实。

### 4.4 已发布问卷保护

第二阶段最容易出错的地方是“复用”和“历史保护”的冲突。系统的解决办法是双层保护：第一层是在问卷导入题目时把定义快照写入 `Survey.questions[]`；第二层是在问卷状态变为 `published` 或 `closed` 后阻止继续编辑。浏览器端会把导入题的标题、描述、类型、选项等字段设为只读，后端也会对已发布问卷的编辑请求进行限制，从而避免题库后续修改污染已发布问卷。

### 4.5 跨问卷统计实现

跨问卷统计不是直接从 `Survey` 中扫描题目文本，而是以 `Response.answers[]` 中的 `sourceQuestionRootId` 为主索引进行聚合。题目详情页因此既能看到该题目被哪些问卷使用，也能看到所有相关问卷中的总体答案分布。这种做法比单纯依赖标题匹配稳定得多，也不会因为题目文案微调而破坏统计口径。

## 5. 关键实现说明

### 5.1 问卷构建器保存链路修复

在第二阶段浏览器级检查中，曾经发现“纯浏览器交互保存问卷后未正确落库”的异常。问题定位后，系统将 `Save Survey` 从依赖原生表单默认提交改为显式按钮流程：前端先统一构建 `builderPayload`，再执行 `reportValidity()`，最后使用 `requestSubmit()` 触发表单提交；`submit` 事件中则保留一次同步，避免 `payload` 丢失。修复后，浏览器路径下的问卷保存、跳转和落库都已经回归通过。

### 5.2 问卷锁定与来源版本提示

在 `builder` 页面中，导入题库题后会明确显示“Imported from question bank root ..., version ...”，并把题目定义字段置为只读，仅允许继续配置问卷级跳转规则。发布后再次打开编辑页时，页面会显示锁定提示，保存按钮消失，标题输入框等关键字段进入禁用状态。这部分行为已经通过浏览器

自动化实际验证。

### 5.3 与第一阶段跳转和校验逻辑的衔接

第二阶段没有重写 `jumpService` 和 `validationService`，而是保持原来的数据驱动方式继续工作。无论题目来自纯自定义创建还是题库导入，最终写入问卷快照后的 `questions[]` 结构都是统一的，因此填写页仍然可以按照原有规则执行 `single_equals`、`multi_contains_any`、`number_between` 等跳转，以及文本长度、数字范围、多选数量等校验。这说明第一阶段的服务层拆分为第二阶段迭代提供了足够支撑。

## 6. AI 使用过程

### 6.1 使用原则

第二阶段仍然遵循“AI 用于整理和辅助，不替代人工验证”的原则。AI 可用于梳理需求变更、生成初步实现思路、帮助定位浏览器交互问题、整理测试文档与报告结构；但涉及字段设计、共享边界、版本链关系、浏览器回归验证和最终报告内容时，均由人工检查、筛选和修正。

### 6.2 AI 使用记录

| 使用阶段   | 使用目的                 | 代表性结果                                |
|--------|----------------------|--------------------------------------|
| 需求理解阶段 | 梳理第二阶段题库、共享、版本链与锁定要求 | 形成增量实现边界和数据模型调整方案。                   |
| 编码实现阶段 | 补充控制器、模型、前端页面与脚本     | 快速完成 Question/QuestionLibrary 等代码骨架。 |
| 调试修复阶段 | 定位浏览器端保存问卷异常         | 修复 builder 保存链路并回归验证。                |
| 文档整理阶段 | 更新 README、测试用例和报告结构  | 统一形成第二阶段交付材料。                        |

### 6.3 AI 带来的帮助

- 帮助把第二阶段需求从自然语言转成可执行的数据模型调整点。
- 帮助快速形成控制器、模型、视图、测试文档的初始版本。
- 帮助定位前端保存链路中的浏览器交互问题。
- 帮助把实现结果整理为 README、测试用例和最终报告。

### 6.4 人工修改与自主完成的部分

系统最终采用的字段命名、版本链组织方式、问卷快照字段、浏览器验证流程、锁定规则和报告表述都经过人工确认。尤其是已发布问卷不可编辑、导入题定义锁定、跨问卷统计口径和浏览器修复回归，必须结合真实代码和运行结果判断，无法仅凭通用答案直接完成。

### 6.5 对 AI 使用的反思

第二阶段更加明显地体现出 AI 的边界：它适合加速信息整理和样板生成，但对“历史一致性保护”“浏览器交互细节”“多集合之间的字段协同”等问题，仍然需要人工理解系统现状后做定向修改。真正有价值的部分不是让 AI 一次性给出所有答案，而是把它当作持续辅助工具，再由开发者完成验证与收束。

## 7. 测试结果

### 7.1 测试方式说明

第二阶段测试以三类方式联合完成：第一，真实启动验证，确认服务和 MongoDB 正常联通；第二，浏览器级手工检查，确认关键页面和交互流程可用；第三，基于《测试用例第二阶段.md》的功能回归，覆盖题目创建、题库管理、共享、版本链、问卷导入、浏览器保存、发布锁定、跨问卷统计以及第一阶段核心功能回归。

### 7.2 主要测试结果汇总

| 序号 | 测试项目     | 结果概述                          | 结论 |
|----|----------|-------------------------------|----|
| 1  | 独立题目创建   | 题目可单独保存并进入详情页，显示 Root ID 和版本号 | 通过 |
| 2  | 题目新版本    | 可基于旧版本创建 v2，并保留父版本关系          | 通过 |
| 3  | 题库创建与加题  | 题库列表和 builder 右侧面板均能看到目标题目    | 通过 |
| 4  | 题目共享     | 共享后其他用户可访问并导入被共享题目            | 通过 |
| 5  | 问卷导入题库题  | 导入后显示来源版本信息，定义字段被锁定           | 通过 |
| 6  | 浏览器端保存问卷 | 修复后可从 New Survey 正常保存并跳转到编辑页  | 通过 |
| 7  | 已发布问卷锁定  | 重新进入编辑页时显示锁定提示且保存按钮消失         | 通过 |
| 8  | 跨问卷统计    | 题目详情页可聚合多问卷回答结果               | 通过 |

### 7.3 典型测试场景说明

场景一为“导入题库题并发布问卷”：用户先创建题目和新版本，再加入题库，随后在问卷构建页导入 v2 题目并发布。结果表明，问卷中成功保存题目快照，填写页和统计页都能正常使用。

场景二为“浏览器保存链路回归”：直接在 New Survey 页面通过真实点击 Save Survey 按钮保存纯自定义问卷。修复前页面跳回 dashboard 但未落库，修复后能够正确进入 /surveys/:id/edit，并在数据库中确认问卷已创建。

场景三为“发布后锁定验证”：发布问卷后再次打开编辑页，系统显示锁定提示，保存按钮不存在，标题输入框为禁用状态。这证明第二阶段的历史保护策略已经在 UI 和后端流程上同时生效。

### 7.4 测试结论

综合测试结果可以看出，第二阶段已经在第一阶段可运行系统的基础上，成功完成题库、共享、版本链、锁定和跨问卷统计等新增能力，并且此前发现的浏览器端保存异常也已经完成修复和回归验证。当前系统已具备作为第二阶段完整成果提交的条件。

## 8. 遇到的问题

### 8.1 题目复用与历史一致性存在天然冲突

如果题目只保留一份最新定义，那么复用确实方便，但旧问卷会被后续修改污染；如果每次导入都完全复制，又会失去共享和版本追踪意义。第二阶段一开始面临的核心难点，就是如何同时支持复用、共享和历史保护。

## 8.2 题目共享会引入额外权限边界

系统需要区分“自己拥有的题目”和“别人共享给我的题目”，还要保证共享不会破坏原拥有者的根版本关系。若权限判断不清晰，可能出现看得到但不能导入、或者导入后来源字段丢失的问题。

## 8.3 浏览器端保存链路与后端逻辑不一致

后端接口曾经可以通过直接 HTTP 请求创建问卷，但真实浏览器点击保存按钮时却没有正确落库。这说明前端交互层存在与后端接口层不同步的问题，也提醒项目不能只做接口级验证。

## 8.4 跨问卷统计口径必须稳定

如果统计只依赖题目标题或某个页面上的文案展示，那么题目改名、复制或版本更新都会影响结果口径。第二阶段必须找到比“显示文本”更稳定的聚合标识。

## 8.5 第二阶段不能推倒第一阶段重做

课程明确要求在已有基础上继续扩展。这意味着任何新增能力都必须兼容旧的填写、跳转、校验与统计流程，否则即使新功能完成，也不算真正满足课程要求。

# 9. 如何解决

## 9.1 通过“独立题目实体 + 问卷快照”同时满足复用与保护

系统采用 Question 集合维护独立版本实体，同时在 Survey.questions[] 中保存导入时的完整快照。这样既能继续复用同一题目的根版本，又能确保旧问卷保留自己的历史内容。

## 9.2 通过 rootQuestionId 和 parentQuestionId 组织版本链

版本链设计把“题目的身份”和“题目的某次修改”分离开来。rootQuestionId 代表同一题目的根身份，parentQuestionId 代表相邻版本之间的演进关系，version 代表面向用户的版本号。这样既方便显示历史，也方便统计根版本的总体使用情况。

## 9.3 通过显式按钮提交流程修复浏览器保存异常

在 builder.js 中新增统一的 payload 组装与同步逻辑后，前端不再依赖脆弱的隐式提交行为，而是通过按钮点击、表单校验、requestSubmit() 的顺序触发提交流程，从而保证浏览器路径和接口路径保持一致。

## 9.4 通过 Response 来源字段保证跨问卷统计稳定

系统把 sourceQuestionId、sourceQuestionRootId、sourceQuestionVersion 写入每条答案记录，使统计聚合建立在稳定的内部标识而不是展示文案上。这样题目标题变更、选项微调或版本迭代都不会破坏统计口径。

## 9.5 通过回归验证保证第一阶段能力不退化

除第二阶段新增功能外，系统还按照测试用例继续覆盖第一阶段的单选/多选/文本/数字题、跳转逻辑、校验逻辑、提交限制和统计功能。只有在这些链路继续可用的前提下，第二阶段扩展才是有效的增量实现。

## 9.6 通过文档同步降低后续维护成本

在功能实现和验证完成后，README、测试用例第二阶段文档和本报告都同步更新为最终版本。这样后续无论是课程验收还是继续扩展功能，都能直接从文档中看出系统现状和边界。

## 10. 总结

综上所述，本项目在第一阶段在线问卷系统的基础上，完成了第二阶段要求的题目独立化、题库管理、题目共享、题目版本链、已发布问卷锁定和跨问卷统计等能力。整个实现过程坚持在原有结构上增量扩展，没有推倒已有系统重写，因而更能体现数据库设计和工程组织的连续性。

第二阶段最有价值的部分并不只是“多了几个页面”，而是通过 Question、QuestionLibrary、Survey、Response 四类集合之间的协作，真正建立起了题目复用与历史保护并存的模型。与此同时，系统还通过真实浏览器验证发现并修复了问卷保存链路问题，使最终提交版本不仅结构上正确，而且在实际操作中能够顺利跑通。

因此，本项目已经较好地完成了课程第二阶段的目标，能够作为《数据库管理系统》课程项目的第二阶段成果提交，并为后续进一步增加版本恢复、图表统计、题库搜索和协作权限等功能打下稳定基础。

代码仓库网站：[https://github.com/Yvon-ld/Database-Lab/tree/main/mongodb\\_survey\\_phase2](https://github.com/Yvon-ld/Database-Lab/tree/main/mongodb_survey_phase2)